

1 Backend (Node/Express + MongoDB)

Folder Structure

```
backend/
├── server.js                # Entry point
├── config/
│   └── db.js                # MongoDB connection
├── models/                  # Database Schemas
│   ├── User.js
│   ├── Company.js
│   ├── NewsArticle.js
│   ├── Category.js
│   ├── Subscription.js
│   ├── Notification.js
│   └── AuditLog.js          # Optional but useful later
├── routes/
│   ├── authRoutes.js
│   ├── userRoutes.js
│   ├── companyRoutes.js
│   ├── newsRoutes.js
│   ├── subscriptionRoutes.js
│   └── notificationRoutes.js
├── controllers/
│   ├── authController.js
│   ├── userController.js
│   ├── companyController.js
│   ├── newsController.js
│   ├── subscriptionController.js
│   └── notificationController.js
├── middlewares/
│   ├── authMiddleware.js
│   ├── errorMiddleware.js
│   └── validateRequest.js
└── utils/
    ├── emailService.js
    ├── generatePDF.js        # For audit/compliance reports
    ├── fetchNewsAPI.js       # For pulling AI/news content
    └── logger.js
```

Database Schemas (Models) Names Only

1. **User** → login, email, password, role (admin/user), preferences
 2. **Company** → name, domain, subscription plan, team members
 3. **NewsArticle** → title, summary, content, category, source, AI_generated (bool), publishedAt
 4. **Category** → name, description
 5. **Subscription** → user/company, plan type, startDate, endDate, active
 6. **Notification** → type, user/company, newsId, status (read/unread)
 7. **AuditLog** → action, performedBy, timestamp (optional but useful for enterprise growth)
-

Route Files Names

- `authRoutes.js` → signup, login, logout, password reset
 - `userRoutes.js` → get/update profile, preferences
 - `companyRoutes.js` → create company, add team members
 - `newsRoutes.js` → fetch AI/news articles, filter by category
 - `subscriptionRoutes.js` → create/update subscription, payment status
 - `notificationRoutes.js` → fetch notifications, mark read/unread
-

Middlewares

- `authMiddleware.js` → JWT validation
 - `errorMiddleware.js` → error handler
 - `validateRequest.js` → request validation (example: Joi)
-

2 Frontend (React + Vite)

Folder Structure

```
frontend/
├── src/
│   ├── main.jsx
│   ├── App.jsx
│   ├── api/
│   │   ├── authApi.js
│   │   ├── userApi.js
│   │   ├── newsApi.js
│   │   └── subscriptionApi.js
│   ├── components/
│   │   ├── Navbar.jsx
│   │   ├── Footer.jsx
│   │   ├── NewsCard.jsx
│   │   ├── CategoryFilter.jsx
│   │   └── NotificationBadge.jsx
│   ├── pages/
│   │   ├── HomePage.jsx
│   │   ├── LoginPage.jsx
│   │   ├── SignupPage.jsx
│   │   ├── Dashboard.jsx
│   │   ├── NewsPage.jsx
│   │   ├── SubscriptionPage.jsx
│   │   └── ProfilePage.jsx
│   ├── context/
│   │   └── AuthContext.jsx
│   ├── hooks/
│   │   └── useFetchNews.jsx
│   ├── styles/
│   │   └── global.css
│   └── utils/
│       └── formatDate.js
```

Frontend Pages / Components (MVP Focus)

MVP Pages

- `HomePage.jsx` → landing page / news feed preview
- `LoginPage.jsx` → login form
- `SignupPage.jsx` → signup form
- `Dashboard.jsx` → main user panel, list of latest AI/news articles
- `NewsPage.jsx` → full news content
- `SubscriptionPage.jsx` → choose/upgrade subscription
- `ProfilePage.jsx` → user profile + preferences

MVP Components

- `NewsCard.jsx` → displays title, summary, category
 - `CategoryFilter.jsx` → filter news by category
 - `NotificationBadge.jsx` → show unread notifications count
-

MVP Scope for 2 Days (Lean)

Focus ONLY on **must-have features for first paying users**:

1. Authentication

- Signup / Login / Logout (AuthNester integration optional)

2. News Feed

- Fetch top 10–20 AI/news articles
- Display summary + link to full content

3. Category Filtering

- Select categories (tech, AI, compliance, etc.)

4. Subscription Plan

- Free / Paid toggle (dummy integration if payment not ready)

5. Notifications

- Simple unread count
- “Mark as read”

6. Profile Page

- Update basic info
- Set preferences

Optional / Later (skip for MVP)

- PDF report generation

- Multi-company support
 - Audit logs
 - Advanced AI summarization / personalization
-

1 Must-Have Frontend Features/Pages (beyond the basics)

1. Trending Section / Page

- Shows the most popular articles (AI-generated + human-curated)
- Could be a **carousel or top 5–10 cards** on the homepage
- Tracks clicks/views → later can feed analytics & personalization

2. Navigation / Navbar Options

- Home
- Trending / Top News
- Categories (Tech, AI, Compliance, etc.)
- Notifications / Alerts
- Profile / Settings
- Subscription / Pricing

3. Search Bar

- Even MVP should allow keyword search in news titles or summaries
- Makes site feel professional and usable

4. User Feedback / Reporting Option

- “Report inaccurate or spammy news” button
- Helps maintain credibility — **essential if AI-generated**

5. Category Filter

- Filters homepage or trending page by category
- Allows users to focus on their niche interest

6. Notification Badge / Panel

- Shows unread notifications (new article in subscribed category)
- Push email notifications optional (later feature)

7. Simple Footer

- Links: About, Contact, Terms, Privacy
 - Optional social media links
-

2 Backend / Database Considerations

1. Article View / Click Tracking

- Add `views` or `clicks` field in `NewsArticle` schema
- Needed for Trending / Popular section

2. Category Popularity

- Store count of clicks per category
- Trending can be calculated by: `most viewed articles last 24h`

3. User Preferences

- Which categories they want to see
- For notifications or personalized feed later

4. Subscription Check

- Certain premium categories or alerts can be gated by subscription

5. Admin / Content Management

- Optional admin route for managing AI-generated news (approve, reject)
 - Helps maintain quality and prevent AI hallucinations
-

3 UX / Engagement Features (even MVP can have them lightly)

- **Highlight AI vs Human Articles** (optional) → builds trust
- **Load More / Pagination** on news pages → prevents long scrolling

- **Timestamp / Source Label** on each article → credibility
 - **Simple Sorting**: Latest first / Trending first
 - **Responsive Design** → mobile-friendly (React + CSS Grid/Flexbox)
-

4 Future-Proofing (without overbuilding)

Even if you skip them for 2-day MVP, **keep these in mind when designing database + frontend components**:

Feature	Why Important
Trending articles / Popular filter	Drives engagement & repeat visits
Categories & Filters	For personalization & subscription targeting
Notifications / Alerts	Monetization & retention potential
Admin approval / moderation	Maintains credibility
Search	Improves stickiness & usability
Click/views tracking	Enables analytics & future AI recommendations
Subscription gating	Revenue & premium content control
User feedback/reporting	Build trust, reduces AI errors liability

5 Recommended MVP Frontend Pages Layout

- HomePage.jsx
 - Trending Section (Top 5 articles)
 - Latest Articles
 - Category Filter
- NewsPage.jsx
 - Full article content
 - Source & timestamp
- TrendingPage.jsx (optional MVP)
 - All popular articles
- LoginPage.jsx / SignupPage.jsx
- Dashboard.jsx
 - Personalized feed (if any)
 - Notifications

- ProfilePage.jsx
 - Preferences & categories
 - SubscriptionPage.jsx
 - Plan info, upgrade option
-

 Reality Tip:

Even if you only build **HomePage + NewsPage + Login/Signup + Dashboard**, you can later **inject Trending Section, Notifications, and Category Filters** without restructuring your backend — just make sure your database tracks:

- views
- category
- publishedAt
- AI_generated (optional)